

Lab 11: React Native Fundamentals

Week 11 Assignment (23.03–29.03.2026)

Student Name: _____ Date: _____

Overview

Total Points: 100 (50 points per lab)

Required Reading: *React and React Native, Fifth Edition* by Sakhniuk & Boduch (Packt 2024)

Chapter 12 (Ch. 11): “Kick-Starting React Native Projects with Expo” (pp. 245–265)

Sections to read: What is React Native? (p. 246), Creating a new Expo project (p. 250), Core Components (p. 254), Styling in React Native (p. 260)

Key Vocabulary

- **React Native:** An open-source framework for building native mobile apps using JavaScript and React (Ch. 12, p. 246).
- **Expo:** A framework and platform for building React Native applications with additional features and simplified workflow (Ch. 12, p. 248).
- **Core Components:** Built-in React Native components like View, Text, and Image that map to native UI elements (Ch. 12, p. 254).
- **StyleSheet:** React Native’s API for creating and managing component styles (Ch. 12, p. 260).
- **Flexbox:** Layout system used in React Native for arranging components (Ch. 12, p. 262).
- **Platform-Specific Code:** Code that behaves differently on iOS vs Android (Ch. 12, p. 256).

React Native vs React (Web)

Aspect	React (Web)	React Native
Renderer	DOM	Native components
Components	div, span, p	View, Text, TextInput
Styling	CSS classes	StyleSheet API
Layout	CSS Flexbox	StyleSheet Flexbox
Events	onClick	onPress
Lists	map()	FlatList, SectionList

Setup Requirements This lab requires:

- Node.js 18+
- npm or yarn

- Expo CLI: `npm install -g expo-cli`
- Expo Go app on your mobile device OR Android Studio/Xcode for emulator
- For physical device: same WiFi network as computer

1. Lab 11.1: Expo Setup and Core Components (50 Points)

Objective

Set up a React Native project using Expo and learn the essential core components. Students will understand the differences between React (web) and React Native, and build their first mobile app screen.

Background

Ch. 12 explains that React Native allows building native mobile apps using JavaScript. Unlike web React that renders to the DOM, React Native renders to native iOS/Android components. Expo simplifies the development process by providing a managed workflow and pre-built native modules.

Core Components (Ch. 12, p. 254):

- **View:** Container component (like div)
- **Text:** Text display (like p, span)
- **Image:** Image display
- **TextInput:** User text input
- **ScrollView:** Scrollable container
- **StyleSheet:** Styling API

Problem Statement

Build a profile screen for a mobile app:

- Create Expo project
- Build profile card with avatar, name, bio
- Add contact information section
- Style using StyleSheet and Flexbox
- Handle user input

Tasks

Task 1: Setup Expo Project (10 points)

1. Create a new Expo project:

```
# Create Expo app (choose "blank" template)
npx create-expo-app@latest ProfileApp --template blank-typescript

# Navigate to project
cd ProfileApp

# Install dependencies
npm install

# Start the development server
npx expo start
```

2. You should see a QR code in the terminal.
3. Scan with Expo Go app on your phone, or press 'a' for Android emulator.
4. Reference: Ch. 12, p. 250 - Creating Expo project.

Task 2: Create Profile Card Component (20 points)

1. Create `src/components/ProfileCard.tsx`:

```
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';

interface ProfileCardProps {
  name: string;
  role: string;
  avatar?: string;
  bio: string;
}

export function ProfileCard({ name, role, avatar, bio }: ProfileCardProps) {
  return (
    <View style={styles.card}>
      <Image
        source={{ uri: avatar || 'https://via.placeholder.com/100' }}
        style={styles.avatar}
      />
      <View style={styles.info}>
        <Text style={styles.name}>{name}</Text>
        <Text style={styles.role}>{role}</Text>
        <Text style={styles.bio}>{bio}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: 'ffffff',
    borderRadius: 16,
    padding: 20,
    margin: 16,
    flexDirection: 'row',
    alignItems: 'center',
    shadowColor: '000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 8,
    elevation: 4,
  },
  avatar: {
    width: 80,
    height: 80,
    borderRadius: 40,
    marginRight: 16,
  },
  info: {
    flex: 1,
  },
  name: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '333',
    marginBottom: 4,
  },
});
```

```
    },
    role: {
      fontSize: 16,
      color: '#666',
      marginBottom: 8,
    },
    bio: {
      fontSize: 14,
      color: '#888',
      lineHeight: 20,
    },
  },
});
```

2. Reference: Ch. 12, p. 254 - Core Components.
3. Note: No className, use StyleSheet instead of CSS.

Task 3: Create Contact Section with Input (20 points)

1. Create src/components/ContactSection.tsx:

```
import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet } from 'react-native';

export function ContactSection() {
  const [email, setEmail] = useState('');
  const [phone, setPhone] = useState('');
  const [message, setMessage] = useState('');
  const [submitted, setSubmitted] = useState(false);

  const handleSubmit = () => {
    if (email && phone && message) {
      setSubmitted(true);
      setTimeout(() => setSubmitted(false), 3000);
    }
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Contact Information</Text>

      <View style={styles.inputGroup}>
        <Text style={styles.label}>Email</Text>
        <TextInput
          style={styles.input}
          value={email}
          onChangeText={setEmail}
          placeholder="your@email.com"
          keyboardType="email-address"
          autoCapitalize="none"
        />
      </View>

      <View style={styles.inputGroup}>
        <Text style={styles.label}>Phone</Text>
        <TextInput
          style={styles.input}
          value={phone}
          onChangeText={setPhone}
          placeholder="+1 234 567 8900"
          keyboardType="phone-pad"
        />
      </View>
    </View>
  );
}
```

```

    </View>

    <View style={styles.inputGroup}>
      <Text style={styles.label}>Message</Text>
      <TextInput
        style={[styles.input, styles.textArea]}
        value={message}
        onChangeText={setMessage}
        placeholder="Your message..."
        multiline
        numberOfLines={4}
        textAlignVertical="top"
      />
    </View>

    <TouchableOpacity
      style={[styles.button, (!email || !phone || !message) && styles.buttonDisabled]}
      onPress={handleSubmit}
      disabled={!email || !phone || !message}
    >
      <Text style={styles.buttonText}>Send Message</Text>
    </TouchableOpacity>

    {submitted && (
      <View style={styles.successMessage}>
        <Text style={styles.successText}>Message sent successfully!</Text>
      </View>
    )}
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    backgroundColor: '#ffffff',
    borderRadius: 16,
    padding: 20,
    margin: 16,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 8,
    elevation: 4,
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 20,
  },
  inputGroup: {
    marginBottom: 16,
  },
  label: {
    fontSize: 14,
    color: '#666',
    marginBottom: 8,
  },
  input: {
    backgroundColor: '#f5f5f5',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,

```

```
    color: '#333',
    borderWidth: 1,
    borderColor: '#eee',
  },
  textArea: {
    height: 100,
    textAlignVertical: 'top',
  },
  button: {
    backgroundColor: '#0066cc',
    borderRadius: 8,
    padding: 16,
    alignItems: 'center',
    marginTop: 8,
  },
  buttonDisabled: {
    backgroundColor: '#ccc',
  },
  buttonText: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  successMessage: {
    backgroundColor: '#d4edda',
    borderRadius: 8,
    padding: 12,
    marginTop: 16,
    alignItems: 'center',
  },
  successText: {
    color: '#155724',
    fontSize: 14,
    fontWeight: '600',
  },
},
});
```

2. Reference: Ch. 12, p. 258 - TouchableOpacity and TextInput.

Deliverable

- Expo project with TypeScript
- `src/components/ProfileCard.tsx`
- `src/components/ContactSection.tsx`
- Updated `App.tsx` with both components
- Screenshot of app running on device/emulator
- README explaining core component differences

Lab 11.1 Assessment Rubric (50 points)

- **Task 1 — Setup (10 pts):** Expo project creates (5), runs on device (5).
- **Task 2 — ProfileCard (20 pts):** View/Text/Image used correctly (10), StyleSheet styling (5), Flexbox layout (5).
- **Task 3 — ContactSection (20 pts):** TextInput handles input (8), TouchableOpacity for button (6), state management (6).

Grading Notes: Using DOM components (div, span): **-20 pts**. No TypeScript: **-5 pts**.

2. Lab 11.2: Lists, Navigation, and Platform-Specific Code (50 Points)

Objective

Learn to work with lists in React Native, implement basic navigation, and handle platform-specific code. These are essential skills for building complete mobile applications.

Background

Ch. 12 explains that mobile apps require different patterns than web apps. Lists in mobile should use `FlatList` for performance, navigation is handled by React Navigation library, and some code may need to be platform-specific.

Key Patterns (Ch. 12):

- `FlatList` for efficient list rendering
- React Navigation for screen navigation
- `Platform.select()` for platform-specific code
- `SafeAreaView` for notch/home indicator handling

Problem Statement

Build a product listing app with navigation:

- Home screen with product list
- Product detail screen
- Navigation between screens
- Platform-specific styling (iOS vs Android)
- Pull-to-refresh functionality

Tasks

Task 1: Create Product List with `FlatList` (15 points)

1. Create `src/data/products.ts`:

```
export interface Product {
  id: string;
  name: string;
  description: string;
  price: number;
  category: string;
  image: string;
}

export const products: Product[] = [
  {
    id: '1',
    name: 'Wireless Headphones',
    description: 'High-quality Bluetooth headphones with noise cancellation',
    price: 199.99,
    category: 'Electronics',
  }
]
```

```

    image: 'https://via.placeholder.com/100',
  },
  {
    id: '2',
    name: 'Smart Watch',
    description: 'Fitness tracker with heart rate monitor',
    price: 299.99,
    category: 'Electronics',
    image: 'https://via.placeholder.com/100',
  },
  {
    id: '3',
    name: 'Running Shoes',
    description: 'Lightweight running shoes with cushioned sole',
    price: 129.99,
    category: 'Sports',
    image: 'https://via.placeholder.com/100',
  },
  {
    id: '4',
    name: 'Coffee Maker',
    description: 'Automatic drip coffee maker with timer',
    price: 79.99,
    category: 'Home',
    image: 'https://via.placeholder.com/100',
  },
  {
    id: '5',
    name: 'Laptop Stand',
    description: 'Ergonomic aluminum laptop stand',
    price: 49.99,
    category: 'Office',
    image: 'https://via.placeholder.com/100',
  },
],
];

```

2. Create src/components/ProductItem.tsx:

```

import React from 'react';
import { View, Text, Image, TouchableOpacity, StyleSheet } from 'react-native';
import { Product } from '../data/products';

interface ProductItemProps {
  product: Product;
  onPress: (product: Product) => void;
}

export function ProductItem({ product, onPress }: ProductItemProps) {
  return (
    <TouchableOpacity style={styles.container} onPress={() => onPress(product)}>
      <Image source={{ uri: product.image }} style={styles.image} />
      <View style={styles.info}>
        <Text style={styles.name}>{product.name}</Text>
        <Text style={styles.category}>{product.category}</Text>
        <Text style={styles.description} numberOfLines={2}>
          {product.description}
        </Text>
        <Text style={styles.price}>${product.price.toFixed(2)}</Text>
      </View>
    </TouchableOpacity>
  );
}

```

```
const styles = StyleSheet.create({
  container: {
    flexDirection: 'row',
    backgroundColor: '#ffffff',
    borderRadius: 12,
    padding: 12,
    marginHorizontal: 16,
    marginVertical: 6,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 2,
  },
  image: {
    width: 80,
    height: 80,
    borderRadius: 8,
    backgroundColor: '#f0f0f0',
  },
  info: {
    flex: 1,
    marginLeft: 12,
    justifyContent: 'center',
  },
  name: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 2,
  },
  category: {
    fontSize: 12,
    color: '#0066cc',
    marginBottom: 4,
  },
  description: {
    fontSize: 13,
    color: '#666',
    marginBottom: 6,
    lineHeight: 18,
  },
  price: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#333',
  },
});
```

3. Reference: Ch. 12, p. 254 - FlatList for lists.

Task 2: Setup React Navigation (20 points)

1. Install React Navigation:

```
npm install @react-navigation/native @react-navigation/native-stack
react-native-screens react-native-safe-area-context
```

2. Create `src/screens/ProductListScreen.tsx`:

```

import React, { useState, useCallback } from 'react';
import { View, FlatList, Text, RefreshControl, StyleSheet, ActivityIndicator } from
  'react-native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { ProductItem } from '../components/ProductItem';
import { products, Product } from '../data/products';

type RootStackParamList = {
  ProductList: undefined;
  ProductDetail: { product: Product };
};

type ProductListScreenProps = {
  navigation: NativeStackNavigationProp<RootStackParamList, 'ProductList'>;
};

export function ProductListScreen({ navigation }: ProductListScreenProps) {
  const [refreshing, setRefreshing] = useState(false);
  const [loading, setLoading] = useState(false);

  const handleProductPress = useCallback((product: Product) => {
    navigation.navigate('ProductDetail', { product });
  }, [navigation]);

  const onRefresh = useCallback(async () => {
    setRefreshing(true);
    await new Promise(resolve => setTimeout(resolve, 1000));
    setRefreshing(false);
  }, []);

  const renderItem = useCallback(({ item }: { item: Product }) => (
    <ProductItem product={item} onPress={handleProductPress} />
  ), [handleProductPress]);

  const keyExtractor = useCallback((item: Product) => item.id, []);

  return (
    <View style={styles.container}>
      <FlatList
        data={products}
        renderItem={renderItem}
        keyExtractor={keyExtractor}
        contentContainerStyle={styles.listContent}
        refreshControl={
          <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
        }
        ListHeaderComponent={
          <Text style={styles.header}>Products ({products.length})</Text>
        }
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
  },
  listContent: {
    paddingVertical: 8,
  },
});

```

```

    header: {
      fontSize: 24,
      fontWeight: 'bold',
      color: '#333',
      marginHorizontal: 16,
      marginVertical: 12,
    },
  });

```

3. Create src/screens/ProductDetailScreen.tsx:

```

import React from 'react';
import { View, Text, Image, ScrollView, TouchableOpacity, StyleSheet, Platform } from
  'react-native';
import { RouteProp } from '@react-navigation/native';
import { Product } from '../data/products';

type RootStackParamList = {
  ProductList: undefined;
  ProductDetail: { product: Product };
};

type ProductDetailScreenProps = {
  route: RouteProp<RootStackParamList, 'ProductDetail'>;
};

export function ProductDetailScreen({ route }: ProductDetailScreenProps) {
  const { product } = route.params;

  return (
    <ScrollView style={styles.container}>
      <Image source={{ uri: product.image }} style={styles.image} />

      <View style={styles.content}>
        <View style={styles.categoryBadge}>
          <Text style={styles.categoryText}>{product.category}</Text>
        </View>

        <Text style={styles.name}>{product.name}</Text>
        <Text style={styles.price}>${product.price.toFixed(2)}</Text>

        <View style={styles.divider} />

        <Text style={styles.sectionTitle}>Description</Text>
        <Text style={styles.description}>{product.description}</Text>

        <View style={styles.platformInfo}>
          <Text style={styles.platformText}>
            Viewing on: {Platform.OS === 'ios' ? 'iOS' : 'Android'}
          </Text>
          <Text style={styles.platformVersion}>
            {Platform.Version}
          </Text>
        </View>

        <TouchableOpacity style={styles.addButton}>
          <Text style={styles.addButtonText}>Add to Cart</Text>
        </TouchableOpacity>
      </View>
    </ScrollView>
  );
}

```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
  },
  image: {
    width: '100%',
    height: 300,
    backgroundColor: '#f0f0f0',
  },
  content: {
    padding: 20,
  },
  categoryBadge: {
    backgroundColor: '#e6f0ff',
    paddingHorizontal: 12,
    paddingVertical: 6,
    borderRadius: 16,
    alignSelf: 'flex-start',
    marginBottom: 12,
  },
  categoryText: {
    color: '#0066cc',
    fontSize: 12,
    fontWeight: '600',
  },
  name: {
    fontSize: 28,
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 8,
  },
  price: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 16,
  },
  divider: {
    height: 1,
    backgroundColor: '#eee',
    marginVertical: 16,
  },
  sectionTitle: {
    fontSize: 18,
    fontWeight: '600',
    color: '#333',
    marginBottom: 8,
  },
  description: {
    fontSize: 16,
    color: '#666',
    lineHeight: 24,
    marginBottom: 20,
  },
  platformInfo: {
    backgroundColor: '#f5f5f5',
    padding: 12,
    borderRadius: 8,
    marginBottom: 20,
  },
  platformText: {
```

```

    fontSize: 14,
    color: '#666',
  },
  platformVersion: {
    fontSize: 12,
    color: '#999',
    marginTop: 4,
  },
  addButton: {
    backgroundColor: '#0066cc',
    borderRadius: 12,
    padding: 16,
    alignItems: 'center',
    ...Platform.select({
      ios: {
        shadowColor: '#0066cc',
        shadowOffset: { width: 0, height: 4 },
        shadowOpacity: 0.3,
        shadowRadius: 8,
      },
      android: {
        elevation: 4,
      },
    }),
  },
  addButtonText: {
    color: '#fff',
    fontSize: 18,
    fontWeight: 'bold',
  },
});

```

4. Reference: Ch. 12, p.256 - Platform.select for platform-specific code.

Task 3: Setup Navigation in App (15 points)

1. Update App.tsx:

```

import React from 'react';
import { StatusBar } from 'expo-status-bar';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import { ProductListScreen } from '../src/screens/ProductListScreen';
import { ProductDetailScreen } from '../src/screens/ProductDetailScreen';
import { Product } from '../src/data/products';

type RootStackParamList = {
  ProductList: undefined;
  ProductDetail: { product: Product };
};

const Stack = createNativeStackNavigator<RootStackParamList>();

export default function App() {
  return (
    <SafeAreaProvider>
      <NavigationContainer>
        <StatusBar style="auto" />
        <Stack.Navigator>
          initialRouteName="ProductList"
          screenOptions={{

```

```
        headerStyle: {
          backgroundColor: '#0066cc',
        },
        headerTintColor: '#fff',
        headerTitleStyle: {
          fontWeight: 'bold',
        },
      }}
    >
    <Stack.Screen
      name="ProductList"
      component={ProductListScreen}
      options={{ title: 'Product Catalog' }}
    />
    <Stack.Screen
      name="ProductDetail"
      component={ProductDetailScreen}
      options={({ route }) => ({ title: route.params.product.name })}
    />
  </Stack.Navigator>
</NavigationContainer>
</SafeAreaProvider>
);
}
```

2. Reference: Ch. 12, p. 252 - React Navigation setup.

Deliverable

- Updated Expo project with navigation
- `src/screens/ProductListScreen.tsx` with `FlatList`
- `src/screens/ProductDetailScreen.tsx`
- `App.tsx` with React Navigation
- Platform-specific styling
- Screenshot of navigation working
- README explaining navigation setup

Lab 11.2 Assessment Rubric (50 points)

- **Task 1 — Product List (15 pts):** `FlatList` with proper keys (8), product item component (7).
- **Task 2 — Navigation Screens (20 pts):** Two screens created (8), navigation between them (6), platform-specific code (6).
- **Task 3 — App Setup (15 pts):** `NavigationContainer` (5), `Stack.Navigator` (5), `screenOptions` (5).

Grading Notes: Using `ScrollView` instead of `FlatList`: **-10 pts**. No navigation: **-15 pts**.

Submission Requirements and Regulations

Git Discipline (30 points)

- **Folder Structure (10 points):** Use `Lab_11/task1/`, `Lab_11/task2/`. Flat dumps: **-10 points**.
- **Conventional Commits (10 points):** Use `feat:`, `fix:`, `refactor:`. Bad messages: **-5 points**.
- **Incremental History (10 points):** At least 3 meaningful commits per task. Single dump: **-10 points**.

Code Quality Standards (20 points)

- **ESLint / TypeScript (10 points):** No TS errors. Major issues: **-5 points**.
- **Documentation (10 points):** README explains React Native vs web. Missing: **-10 points**.

AI Usage Policy

If you use AI tools:

- Submit `AI_REPORT` with: tool used, prompts, how you modified/verify code, what you learned.
- Missing report when AI used: **-50 points**. Incomplete report: **-20 points**.

Submission Instructions

1. **Lab 11.1:** Expo setup + Profile Card + Contact Section.
2. **Lab 11.2:** Product List + Navigation + Platform-specific code.
3. Use `Lab_11/task1/`, `Lab_11/task2/` folder structure.
4. Include screenshots of app running.
5. Include name, student ID, date in README files.
6. Submit via course portal.

Comprehensive Assessment Summary

Category	Points	Assessment
Lab 11.1 Tasks	50	See Lab 11.1 Rubric
Lab 11.2 Tasks	50	See Lab 11.2 Rubric
Git: Folder structure	10	[YES] / [NO] (-10)
Git: Conventional commits	10	[YES] / [NO] (-5)
Git: Incremental history	10	[YES] / [NO] (-10)
Code quality	10	[YES] / [NO] (-5)
Documentation	10	[YES] / [NO] (-10)
TOTAL	100	

Appendix A: React Native Core Components Reference

View Component (Ch. 12, p. 254):

- Fundamental container for grouping other components
- Maps to native `UIView` (iOS) or `ViewGroup` (Android)
- Supports Flexbox layout
- Does not have visual appearance by default

```
<View style={styles.container}>
  <View style={styles.header} />
  <View style={styles.content} />
  <View style={styles.footer} />
</View>
```

Text Component:

- Displays text content
- Must be nested inside `Text` for mixed styling
- Supports nested `Text` for inline styles

```
<Text style={styles.title}>
  Welcome to <Text style={styles.bold}>React Native</Text>
</Text>
```

Image Component:

- Displays images from various sources
- Requires explicit width and height
- Supports remote URLs and local assets

```
// Remote image
<Image source={{ uri: 'https://example.com/image.png' }} style={styles.image} />

// Local asset
<Image source={require('./assets/icon.png')} style={styles.icon} />
```

TextInput Component (Ch. 12, p. 258):

- User text input field
- Supports various keyboard types
- Has `onChangeText` callback for value changes

```
<TextInput
  style={styles.input}
  value={text}
  onChangeText={setText}
  placeholder="Enter text..."
  keyboardType="email-address"
  secureTextEntry={false}
  autoCapitalize="none"
  multiline
/>
```

Touchable Components (Ch. 12, p. 258):

Component	Use Case
TouchableOpacity	Buttons with opacity feedback
TouchableHighlight	Buttons with highlight feedback
TouchableWithoutFeedback	When you don't want any feedback
Pressable	Modern alternative with more control

ActivityIndicator:

- Shows a loading spinner
- Use when loading data or processing

```
<ActivityIndicator size="large" color="#0066cc" />
```

Appendix B: StyleSheet and Flexbox in React Native

StyleSheet.create() (Ch. 12, p. 260):

- Creates stylesheet from style objects
- Validates style properties at creation time
- Improves performance by avoiding style recreation
- Provides type checking for style properties

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'fff',
    padding: 16,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#333',
  },
});
```

Flexbox in React Native (Ch. 12, p. 262):

Container Properties:

Property	Description
flexDirection	'row' 'column' (default)
justifyContent	'flex-start' 'center' 'flex-end' 'space-between' 'space-around'
alignItems	'flex-start' 'center' 'flex-end' 'stretch'
flexWrap	'nowrap' 'wrap'
flexGrow	Number - how much to grow
flexShrink	Number - how much to shrink

Item Properties:

Property	Description
alignSelf	Override alignItems for this item
flex	1 means fill available space
margin	Spacing outside component
padding	Spacing inside component

Common Layout Patterns:

```
// Center content
<View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
  <Text>Centered</Text>
</View>
```

```
// Row with space between
<View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
  <Text>Left</Text>
  <Text>Right</Text>
</View>

// Flexible list item
<View style={{ flex: 1 }}>
  <Text style={{ flex: 1 }}>Takes remaining space</Text>
</View>
```

Platform-Specific Styles:

```
<View style={styles.container}>
  <Text style={[
    styles.text,
    Platform.OS === 'ios' ? styles.iosText : styles.androidText
  ]}>
    Platform-specific
  </Text>
</View>

// Using Platform.select
const padding = Platform.select({
  ios: 20,
  android: 16,
});
```

Appendix C: React Navigation Quick Reference

Installation (Ch. 12, p. 252):

```
npm install @react-navigation/native @react-navigation/native-stack
react-native-screens react-native-safe-area-context
```

Basic Stack Navigator:

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

type RootStackParamList = {
  Home: undefined;
  Details: { id: string };
};

const Stack = createNativeStackNavigator<RootStackParamList>();

function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Navigation Props:

```
// For typed navigation props
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { RouteProp } from '@react-navigation/native';

type DetailsScreenProps = {
  navigation: NativeStackNavigationProp<RootStackParamList, 'Details'>;
  route: RouteProp<RootStackParamList, 'Details'>;
};

// Using navigation
function DetailsScreen({ navigation, route }: DetailsScreenProps) {
  const { id } = route.params;

  return (
    <Button onPress={() => navigation.goBack()} title="Go Back" />
  );
}
```

Stack Navigator Options:

```
<Stack.Screen
  name="Details"
  component={DetailsScreen}
  options={{
    title: 'Product Details',
    headerStyle: { backgroundColor: '#0066cc' },
    headerTintColor: '#fff',
    headerTitleStyle: { fontWeight: 'bold' },
```

```
      headerBackTitle: 'Back',  
    }  
  />
```

Common Navigation Methods:

Method	Description
navigation.navigate('Screen')	Go to screen
navigation.goBack()	Go back
navigation.push('Screen')	Push new screen
navigation.popToTop()	Go to first screen
navigation.replace('Screen')	Replace current screen